

Пошук горіхів

Ліміт часу: 8 s Ліміт пам'яті: 256 MiB

Ви шукаєте давно загублений горіх-скарб на величезному полі клітин розміром $N \times N$ за допомогою чарівної білки, на ім'я Ілля. Усього на полі заховано $K \leq 3$ скринь із горіхами в різних клітинах. Ваша мета проста: знайти їх усі!

Коли ви ставите білку на одну з клітин, вона знаходить найкоротші шляхи до якоїсь скрині, використовуючи лише переміщення ліворуч, угору, праворуч і вниз (тобто до найближчої скрині за Манхеттенською відстанню). Потім вона показує напрямок першого кроку. Якщо існує кілька можливих перших кроків, що ведуть до якоїсь найближчої скрині (або до кількох таких скринь), білка поверне всі ці напрямки. Якщо клітина містить скриню, білка вкаже на це замість напрямку.

Щоразу, коли ви знаходите скриню з горіхом, ви спорожняєте її вміст, але не можете прибрати саму скриню — вона надто важка. Ваша білка не знає, порожня скриня чи повна. Білка показуватиме шлях до найближчої скрині незалежно від того, порожня вона чи повна.

Спробуйте знайти розташування всіх скринь із горіхом, використовуючи якомога менше запитів до білки.

Задача

Це інтерактивна задача. У кожному тестовому випадку (тобто під час кожного запуску) програма повинна буде розв'язати кілька пошуків горіхів. Для взаємодії з перевіряючою системою слід використовувати бібліотеку, надану організаторами. Ця бібліотека містить такі оголошення:

- **void** *NextHunt*(**int** &*N*, **int** &*K*) — викликайте цю функцію, щоб розпочати наступний пошук горіхів. Вона задасть розмір сітки у змінній *N* та кількість горіхів у *K*. Якщо в поточному запуску більше немає пошуків горіхів, які потрібно розв'язати, функція встановить *N* і *K* в -1 ; у цьому випадку слід завершити програму з кодом виходу 0 . Зверніть увагу, що ви можете викликати цю функцію ще до того, як знайдете усі горіхи поточного пошуку, наприклад, якщо намагаєтеся отримати лише часткові бали.
- **enum** {*TREASURE* = 0 , *DIR_RIGHT* = 1 , *DIR_UP* = 2 , *DIR_LEFT* = 4 , *DIR_DOWN* = 8 }; — це константи, які використовуються у значеннях, що повертає функція *Query* (див. нижче).
- **int** *Query*(**int** *x*, **int** *y*) — якщо клітина з координатами (*x*, *y*) містить скриню, ця функція повертає *TREASURE*. Інакше вона повертає суму однієї або кількох констант *DIR_RIGHT*, *DIR_UP*, *DIR_LEFT* і *DIR_DOWN*, вказуючи, які переміщення повертає білка, коли її розміщено в клітині (*x*, *y*). Координати *x* і *y* повинні бути цілими числами від 0 до $N - 1$. (Примітка: у цій задачі координати *y* зростають зверху вниз.)

Після того як *NextHunt* присвоє $N = K = -1$, ваша програма більше не повинна викликати ні *NextHunt*, ні *Query*. Вона також не повинна викликати *Query* до

першого виклику *NextHunt*. Якщо програма не дотримається цих вимог, або якщо вона виконає понад 1000 запитів під час одного пошуку горіхів, або якщо передасть у *Query* значення x та/або y поза дозволеним діапазоном, бібліотека завершить роботу програми та поверне вердикт помилка виконання (RTE) для поточного тестового випадку.

Горіх вважається знайденим, якщо ви хоча б один раз виконали запит до клітини, яка його містить. Якщо програма викликає *NextHunt* до того, як знайде всі горіхи поточного пошуку, це не вважається помилкою, але вплине на результат (детальніше про оцінювання нижче).

Для доступу до бібліотеки програма повинна включити заголовний файл `treasurehuntlib.h`:

```
#include "treasurehuntlib.h"
```

Цей файл можна завантажити тут: `treasurehuntlib.h`.

Для допомоги в розробці рішення доступна проста реалізація бібліотеки: `treasurehuntlib-public.cpp`. Аби скомпілювати її разом зі своєю програмою, просто додайте її ім'я як аргумент компілятора, наприклад:

```
g++ foo.cpp treasurehuntlib-public.cpp
```

де `foo.cpp` — це ім'я файлу, що містить ваше рішення.

Реалізація у `treasurehuntlib-public.cpp`, окрім наведених вище оголошень, також підтримує ще одну функцію, `void InitFromFile(const char *fileName)`, яка зчитує список конфігурацій пошуку горіхів з файлу та дозволяє грати в гру саме з ними замість випадково згенерованих конфігурацій, які бібліотека створює самостійно. Докладнішу інформацію можна знайти безпосередньо у файлі `treasurehuntlib-public.cpp`.

На сервері перевірки буде використовуватися інша реалізація бібліотеки, тому не слід робити жодних припущень щодо того, як саме вона працює. Водночас можна вважати, що вона не засмічує глобальний простір імен жодними оголошеннями, окрім наведених вище (*NextHunt*, *Query* та п'яти констант).

Ваша програма не повинна читати зі стандартного вводу чи записувати у стандартний вивід, оскільки вони будуть використовуватися реалізацією бібліотеки на сервері перевірки для зв'язку з рештою середовища оцінювання.

Вхідні дані

Обмеження

- $1 \leq N \leq 10^6$
- $1 \leq K \leq 3$
- Під час одного запуску програми буде не більше ніж 100 000 пошуків горіхів.
- Під час одного пошуку горіхів можна виконати не більше ніж 1000 запитів.
- Система оцінювання не адаптивна.

Підзадачі

- Підзадача 1 (10 балів) $K = 1$
- Підзадача 2 (30 балів) $K = 2$
- Підзадача 3 (60 балів) $K = 3$.

Оцінювання

Підзадача може складатися з кількох тестових випадків (кількох запусків програми), а кожен тестовий випадок може містити кілька пошуків горіхів. Для цілей оцінювання всі пошуки в межах однієї підзадачі оцінюються разом, незалежно від того, як вони були розподілені між тестовими випадками. Для i -го пошуку горіхів позначимо розмір поля як $N_i \times N_i$, кількість горіхів як K_i , кількість запитів, виконаних програмою, як Q_i , а кількість знайдених горіхів як F_i . Також нехай S — загальна кількість балів, доступних за цю підзадачу. Тоді кількість балів, отриманих програмою за цю підзадачу:

- Якщо програма завжди знаходила всі горіхи (тобто якщо $F_i = K_i$ для всіх i), її бали залежать від $t_i = \frac{Q_i}{\lceil \log_2 N_i \rceil}$

$$\frac{S}{2} + \frac{S}{2} \cdot \min_i f(t_i) \text{ балів, де } f(t_i) = \begin{cases} 1, & t_i \leq 11 \\ 1 - (t_i - 11)/9, & 11 \leq t_i \leq 20 \\ 0, & t \geq 20. \end{cases}$$

- Якщо програма не завжди знаходила всі горіхи (тобто існує i , для якого $F_i < K_i$), вона отримує

$$\frac{S}{2} \cdot \min_i \frac{F_i}{K_i} \text{ балів.}$$

Іншими словами, половину балів ви отримуєте за те, що знаходите усі горіхи, а іншу половину — за те, що знаходите їх, використовуючи якомога менше запитів. Для максимального результату ваше рішення повинно знаходити всі горіхи не більше ніж за $11 \lceil \log_2 N_i \rceil$ запитів. Між $11 \lceil \log_2 N_i \rceil$ і $20 \lceil \log_2 N_i \rceil$ запитами оцінка лінійно зменшується; якщо ж рішення виконає більше ніж $20 \lceil \log_2 N_i \rceil$ запитів, воно отримає лише першу половину балів за знаходження всіх горіхів. (Символи $\lceil \cdot \rceil$ означають, що значення $\log_2 N_i$ округлюється вгору до найближчого цілого числа.)

Якщо наведені вище формули дають нецілу кількість балів за підзадачу, результат округлюється до найближчого цілого числа.

Якщо програма завершиться з помилкою виконання або не дотримуватиметься описаного вище протоколу використання бібліотеки, вона отримає 0 балів за всю підзадачу. Щоб отримати часткові бали за знаходження лише підмножини горіхів, програма повинна коректно завершити пошук, викликавши *NextHunt*.

Приклад

Виклик	Повернене значення
NextHunt(N, K)	$N = 4, K = 1$
Query(2, 0)	$DIR_DOWN + DIR_RIGHT = 9$
Query(3, 1)	DIR_DOWN
Query(3, 2)	$TREASURE$
NextHunt(N, K)	$N = -1, K = -1$